

Online Voting System

Python Mini Project using OOP, Arrays and Validation

STUDENT NAME

Lakshya Sonwal

ROLL NUMBER

0834CS241096

COLLEGE

Sushila Devi Bansal College of Engineering

DEPARTMENT

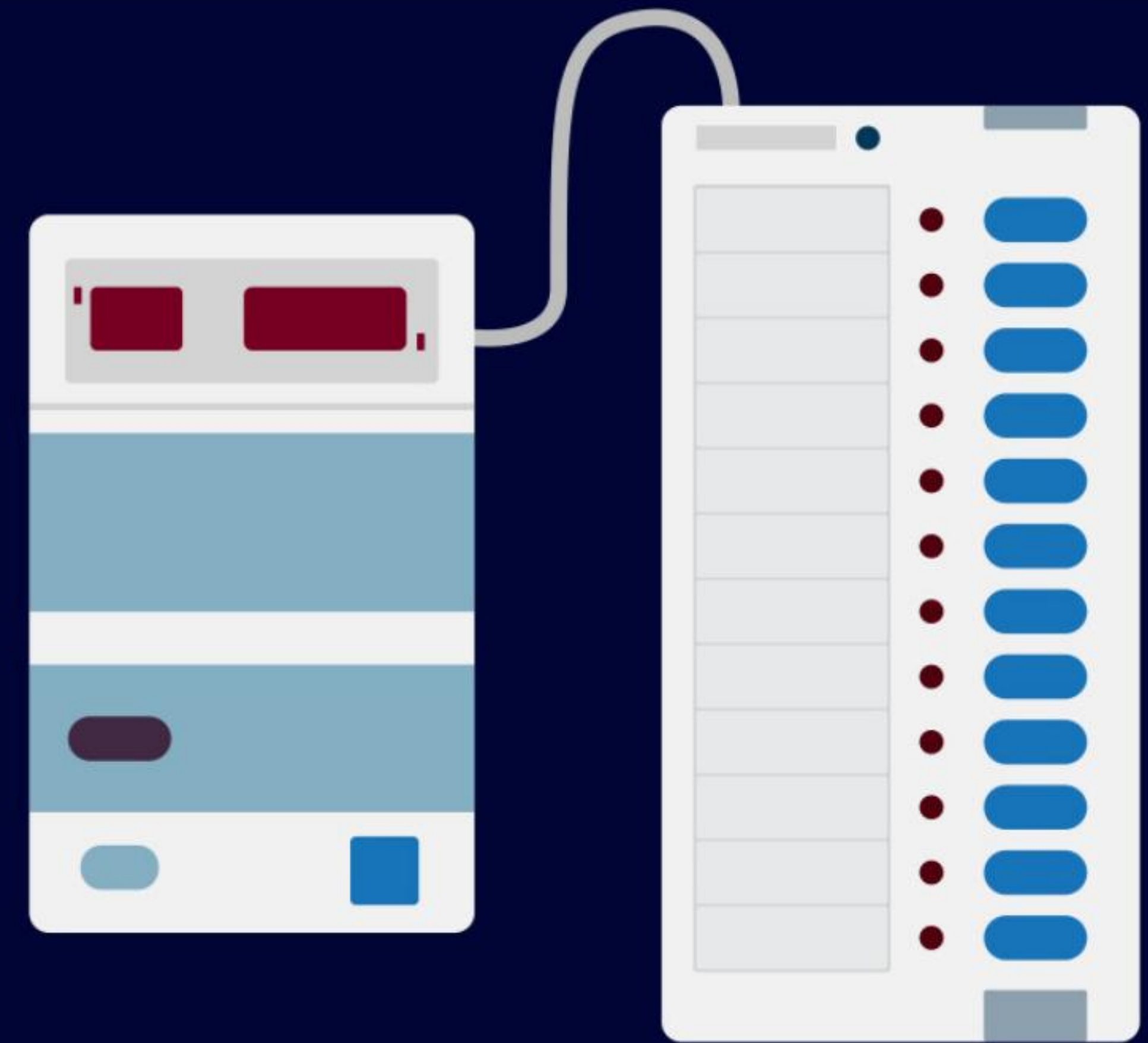
Computer Science & Engineering

| INTRODUCTION

An **Online Voting System** is an advanced, digital platform engineered to enable eligible electors to securely cast their votes over a network.

By replacing archaic paper ballots with code, digital voting streamlines convenience, cuts down logistical waste, and enables high-speed automated auditing.

The foundation of modern democracy rests on secure, highly transparent, and universally accessible voting ecosystems.



| PROJECT OBJECTIVES



Identity Protection

Secure login modules and precise registration workflows to validate voter credentials before access.



Anti-Fraud Logic

Execution of boolean states to eliminate duplicate voting, ensuring each user receives only one vote.



Automated Counting

Instant vote calculation and final election analytics constructed safely using Python's OOP practices.

MANUAL VOTING LIMITATIONS

⚠ The Issues

Manual workflows are prone to systematic blocks, including:

- ✗ **Time-Consuming:** Lengthy polling booth queues and printing.
- ✗ **Human Errors:** Manual counting leads to inaccuracies.
- ✗ **Security Vulnerabilities:** Stolen ballots and duplication risks.

💡 The Need for Transition

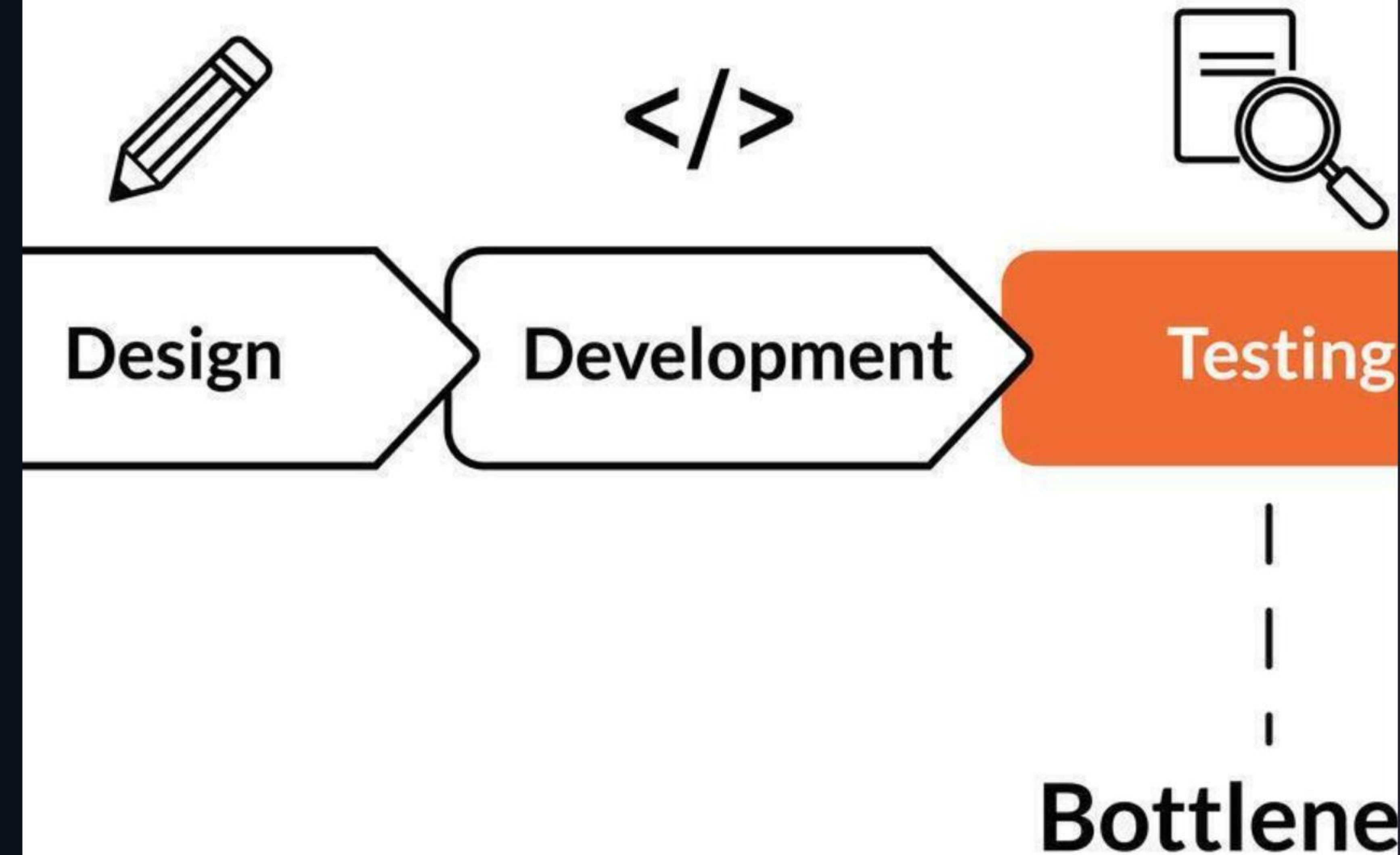
Our digital shift aims to make physical validation systems obsolete by introducing linear software checks, guaranteeing tamper-proof audit trails for both candidates and voters.

PROPOSED SOLUTION

We propose a Python-driven **Console Voting Application** structured on standard Object-Oriented principles.

By encapsulating data within robust classes like `Voter`, `Candidate`, and `Election`, the system acts as an isolated sandbox for testing democracy rules safely.

Ensures real-time calculations without third-party databases, executing flawlessly within minimal hardware environments.



| SYSTEM FEATURES

-  **Voter Registration:** Captures age, name, and assigns a unique system ID.
-  **Secure Login:** Dynamic runtime credential evaluation.
-  **Anti-Double Vote:** Toggle check to flag `voted_status = True` immediately.
-  **Real-time Tally:** Background loops calculate candidate counts instantly.
-  **Automated Winner:** Instant comparison of metrics on election conclusion.
-  **Interactive CLI:** Straightforward terminal navigation interface.

| TECHNOLOGIES & DESIGN CONCEPTS



Python 3.x Engine

The core computational language utilized to power arrays and class logic.



OOP Structure

Uses encapsulation to split voters, candidates, and election tasks into cleanly defined blocks.



Logic Arrays

Stores lists containing user records and candidate objects dynamically in local memory.

SYSTEM FLOWCHART



This linear logical flow ensures that no voter can access the voting pool without satisfying strict validation algorithms.

UML ARCHITECTURE (CLASSES)

Class Name	Properties (Data)	Methods (Logic Actions)
Voter	id , name , age , has_voted	register() , login() , verify_age()
Candidate	candidate_id , party_name , votes	increment_count() , get_metrics()
Election	voter_db , candidate_db	execute_session() , tabulate_winner()

PROGRAM EXECUTION INTERFACE

cli_voter_registration.py

>>> Running voter registration...

Enter voter name: Lakshya Sonwal

Enter voter age: 20

Success: Registration complete. ID allocated: VT-1096

cli_vote_cast.py

>>> Logged in: VT-1096

1. Candidate A | 2. Candidate B

Select Candidate choice [1-2]: 1

Success: Vote recorded. Voter status flagged: VOTED.

| KEY ADVANTAGES

Our prototype represents a reliable model of high-security voting automation.

100%

Fraud Resistant

0.0s

Tally Time

-  **Robust Validation:** Strict age and status validations.
-  **Zero Latency:** No manual counting errors or downtime.
-  **Academic Value:** Demonstrates real-world OOP workflows.

| FUTURE SCOPE



Database Integration

Scale local arrays to a secure, permanent storage system like MySQL or SQLite.



OTP Validation

Deploy secure verification via dynamic email OTP systems for identity security.



Tkinter Graphical UI

Transition CLI output into modern layouts using standard desktop GUI frames.

Project Conclusion

This online voting project successfully prototypes a secure voting environment, showing how **OOP principles** translate into stable administrative software solutions.

Python 3 • Memory Arrays • Status Flags • Object Encapsulation

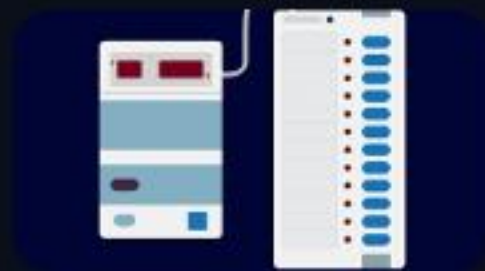
Thank You

Open for any queries, reviews, or technical questions.

Lakshya Sonwal

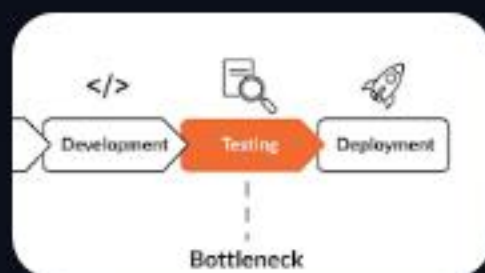
CSE | SUSHILA DEVI BANSAL COLLEGE OF ENGINEERING

IMAGE SOURCES



<https://cdn.vectorstock.com/i/1000v/22/26/electronic-voting-machine-for-election-flat-vector-49732226.jpg>

Source: www.vectorstock.com



https://static.vecteezy.com/system/resources/previews/077/780/347/non_2x/process-diagram-highlighting-bottleneck-in-software-lifecycle-vector.jpg

Source: www.vecteezy.com